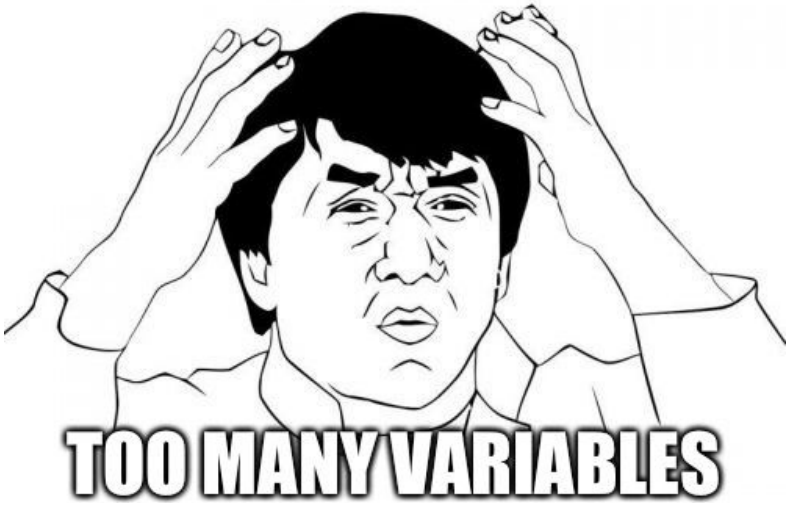
programando com python

etapa 1: várias variáveis

& alguns operadores &
métodos &
mais

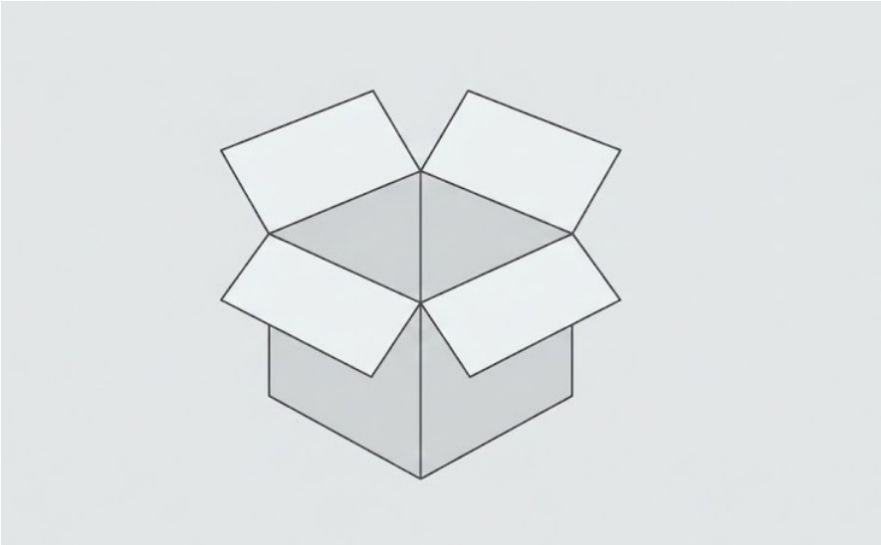


- ➔ variáveis
- ➔ tipos em python
- ➔ métodos
- ➔ type casting
- ➔ interações com o terminal
- ➔ operadores aritméticos

variáveis em python são fáceis

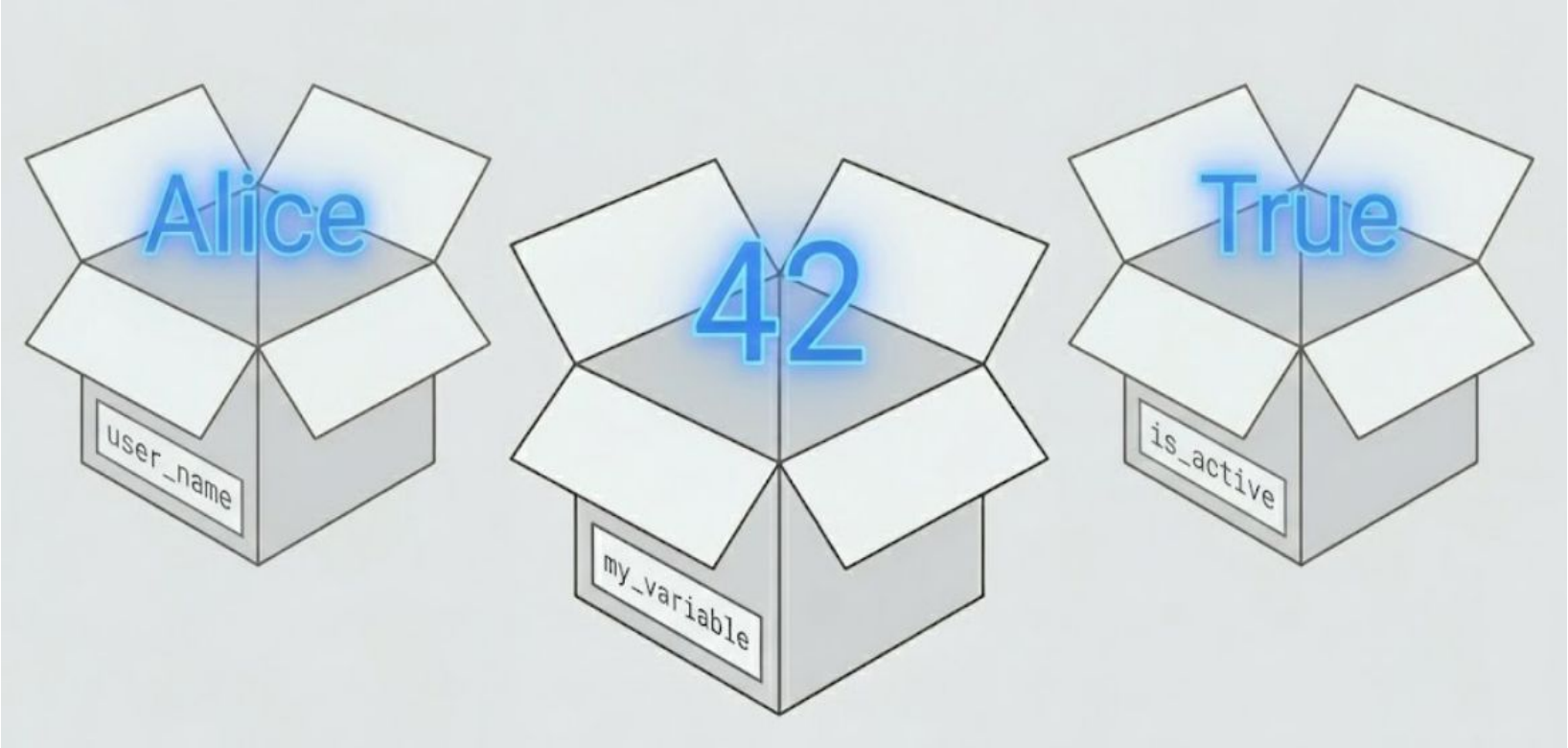
pense nelas como caixas que armazenam **um** valor, pra isso precisamos **nomear** essa caixa e **atribuir** um valor.

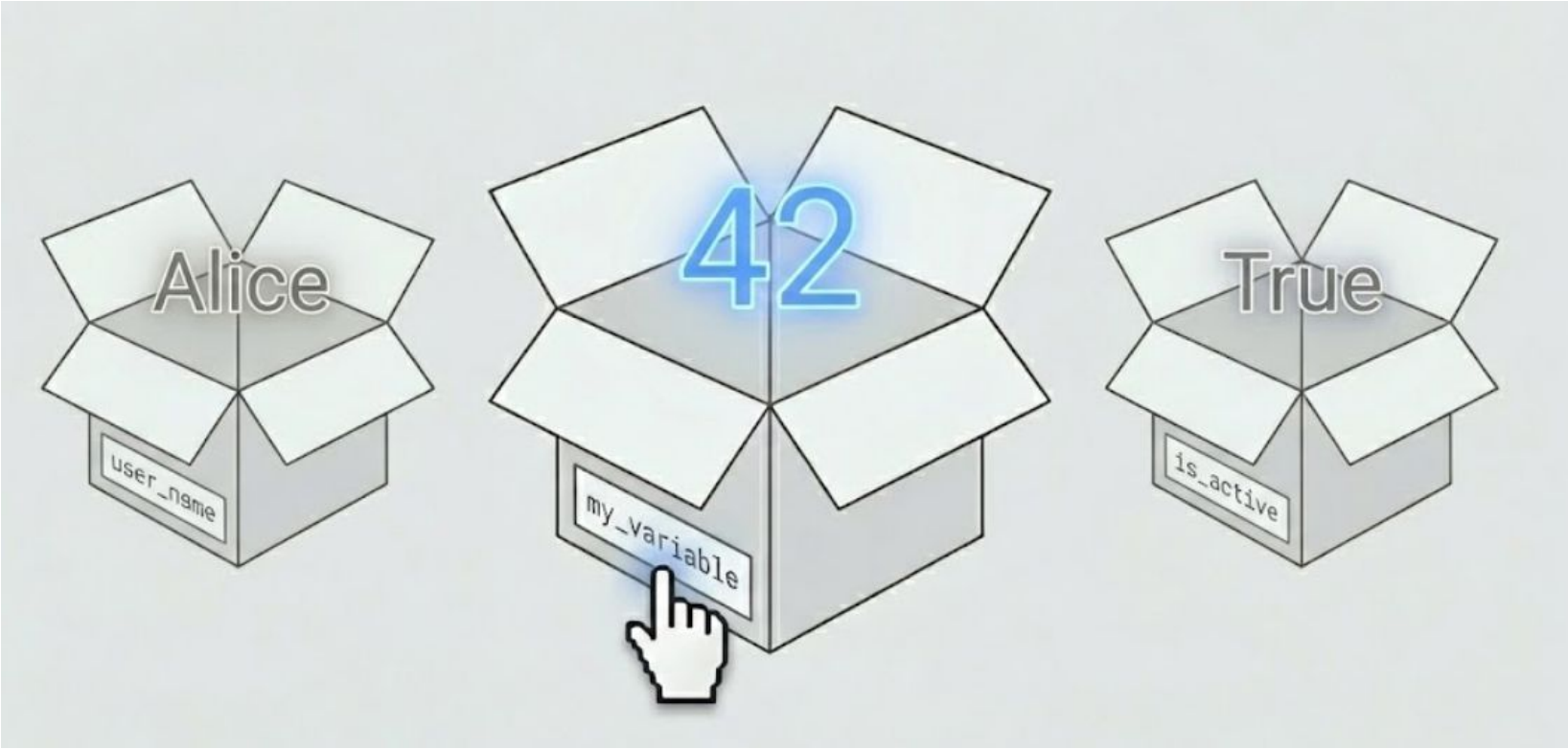
o nome ajuda a encontrar esse valor para ser utilizado novamente depois.



```
my_variable = 42
```

42
my_variable


Alice
user_name
42
my_variable
True
is_active


Alice
user_name
42
my_variable
True
is_active

coisas importantes sobre variáveis

- ➔ o nome **não** deve iniciar com um números (ex: 1nome ❌ / nome1 ✅)
- ➔ o nome **não** deve iniciar com espaços ou caracteres especiais (ex: #nome ❌, !nome ❌, `nome`❌)
- ➔ as palavras reservadas que vimos antes **não** podem nomear uma variável!
- ➔ em python é uma boa prática utilizar o **padrão snake_case** para nomear coisas (ex: meu_nome, minha_variavel)
- ➔ variáveis que não se alteram devem ser nomeadas em letras **maiúsculas.**
- ➔ tudo em python é ***case sensitive*,** letras maiúsculas e minúsculas fazem diferença!
- ➔ o valor armazenado em uma variável vai definir os **tipos de operação** que podem ser feitas com ela.
- ➔ em python o tipo de uma variável pode ser alterado, isso acontece pois se trata de uma linguagem **com tipagem dinâmica** (na prática estamos criando um novo objeto em memória).
- ➔ lembre-se que a leitura do código acontece **sequencialmente**, linha após linha. uma mudança posterior não vai afetar linhas anteriores.
- ➔ nomear as variáveis com **clareza** é um passo importante para um código melhor.

declarando variáveis

dá uma olhada no código

repare que a ide avisa dos problemas encontrados em todos as declarações incorretas.

observe também os diferentes valores que podem ser armazenados em uma váriavel, desde uma linha de texto até palavras reservadas.

a formatação de entrada desse valor vai definir o seu **tipo.**

```
# nomeclatura incorreta de variáveis em python

1nome = "João"  # não pode começar com número
 nome = "Maria"  # não pode começar com espaço
idade anos = 20  # não pode conter espaço
minha-idade = 25  # não pode conter hífen
$salario = 3000.48  # não pode conter símbolo de cifrão
idade! = 30  # não pode conter ponto de exclamação
for = 10  # não pode conter palavra reservada

# declarando variáveis corretamente

nome = "jimmy page"
idade = 81
altura = 1.8
vivo = True
em_atividade = False
```

enxergando dentro da caixa

crie uma variável qualquer e insira um valor para ela, pode ser um texto, número ou uma afirmação do tipo booleana (True ou False).

agora precisamos verificar se o valor foi armazenado corretamente na caixa criada, usamos a função do python chamada **print.** uma função nativa do python que vai nos ajudar a interagir com nosso código a partir do terminal.

a função print exige que seja passado um **parâmetro**, isso acontece inserindo o valor dentro dos parênteses, em seguida exibe o valor passado no seu terminal.

anteriormente utilizamos a função enviando um valor diretamente, dessa vez tente chamar a função e **passe como parâmetro a sua variável**.

observe o se o resultado no seu terminal é equivalente ao que foi atribuído anteriormente.

interagindo_com_variaveis.py > ...

```
1   minha_banda_favorita = "tião carreiro e pardinho"
2   print(minha_banda_favorita)
3
4
```

print()

crie outras variáveis relacionadas a criada anteriormente, utilize a função print() para criar uma saída no terminal que informe os valores de cada uma delas, pode ser uma mensagem, um relatório ou simplesmente informações sequenciais.

você pode passar mais de um parâmetro separando eles com **vírgula**, misturando texto e números por exemplo. ou chamar a função print() para criar uma linha de separação.

```
minha_musica_favorita = "sweetest chill"
print("minha música favorita:")
print(minha_musica_favorita)

ano_de_fundacao = 1976
print("ano de fundação:", ano_de_fundacao)
```

```
minha música favorita:
sweetest chill
ano de fundação: 1976
```

os parâmetros também podem ser misturados com operadores aritméticos

```
print("- ● -" * 10)

minha_musica_favorita = "sweetest chill"
print("minha música favorita:")
print(minha_musica_favorita)

print("- ● -" * 10)

ano_de_fundacao = 1976
print("ano de fundação:", ano_de_fundacao + 1)
```

```
- ● -- ● -- ● -- ● -- ● -- ● -- ● -- ● -- ● -- ● -
minha música favorita:
sweetest chill
- ● -- ● -- ● -- ● -- ● -- ● -- ● -- ● -- ● -- ● -
ano de fundação: 1977
```

formatação de strings

a forma mais moderna e eficiente de exibir informações provenientes de variáveis através do print(), é usando uma **string formatada**. uma string é o nome dado do tipo de variável que armazena um texto, você tem visto ele com frequência, é normalmente o tipo usado como parâmetro quando chamamos a função print().

uma string formatada permite que você misture lógica de programação dentro do texto estático de uma string.

a representação de uma string formatada é o uso da letra ***f*** antes de abrir as aspas e envolvendo a variável ou lógica em ***{ chaves }***:

```
# string formatada
print(f"banda favorita: {minha_banda_favorita}")
```

```
a = 1
b = 666
print(f'{a} + {b} = {a + b}')
```

```
banda favorita: siouxsie and the banshees
1 + 666 = 667
```

formatando strings - quebrando linhas

você pode quebrar linhas sem a necessidade de chamar novamente a função print() usando ***\n*** durante o texto.

```
# string formatada com quebra de linha
print(f"ano de fundação:\n{ano_de_fundacao}\nmusica favorita:{minha_musica_favorita}")
```

```
ano de fundação:
1976
musica favorita:sweetest chill
```

também é possível fazer com que a formatação do texto seja seguida no print envolvendo o texto com ***'''*** da mesma forma que o comentário em bloco.

```
# string formatada com quebra de linha por formatacao
print(f'''ano de fundação: {ano_de_fundacao}
      musica favorita:{minha_musica_favorita}''')
```

```
ano de fundação: 1976
      musica favorita:sweetest chill
```

print sem paramêtro também quebra linha :p

```
# print vazio pulando linha
print("inicio do programa")
print()
print("fim do programa")
```

```
inicio do programa

fim do programa
```

formatando strings - arredondando valores

você pode formatar prints com tabulação usando ***\t,*** embora seja pouco usado devido a bibliotecas que fazem um trabalho melhor.

também é possível arredondar valores do tipo **float** usando :.{ quantidade de casas }f

```
# print tabulado com \t
print("itens:\n\t- item 1\n\t- item 2\n\t- item 3 \t- item 4")

# print formatado com casas decimais
pi = 3.14159265359
print(f'Valor de pi: {pi:.4f}')
```

```
itens:
        - item 1
        - item 2
        - item 3        - item 4

Valor de pi: 3.1416
```

tipos de variáveis

por ser uma linguagem que busca simplificar as coisas, temos apenas 4 categorias principais em python:

str (string): textos, sempre formatados entre aspas, sejam elas simples ou duplas (`“olá, mundo”, ‘olá mundo’`)

int: números inteiros positivos ou negativos, em python não existe limite no tamanho. (100, -198, 0)

float: números decimais positivos ou negativos utilizando . e não , (3.14, -1.77)

boolean: valor binário oriundo da algebra de boole, pode ser **True** ou **False** (sempre com letra maiúscula.)

a função **type()** pode ser usada para verificar o tipo de uma variável, ela retorna o tipo e pode ser passada como paramêtro na função print().

```
# string
nome = "pedro"
tipo_da_variavel = type(nome)
print(tipo_da_variavel)
# int
idade = 24
print(type(idade))
```

```
<class 'str'>
<class 'int'>
```

operadores aritméticos

atribuição: = atribui um valor.

soma: + soma dois valores.

subtração: - subtrai dois valores.

multiplicação: * multiplica dois valores.

exponenciação: ** eleva um valor a potência do outro.

divisão simples: / divide um valor pelo outro e retorna o resultado decimal.

divisão absoluta: // divide um valor pelo outro e retorna o resultado inteiro.

divisão módulo: % divide o valor pelo outro e retorna o resto da operação.

- os operadores podem ser combinados com o operador de atribuição para formar operadores compostos, mais simples de usar:
  - soma: **+=**
  - subtração: **-=**
  - divisão: **/=**
  - multiplicação: ***=**

strings

a tradução seria algo como linha ou fio, um texto é formado de diversos caracteres formando uma string. mas uma string vai além de apenas texto, são objetos que possuem suas características próprias, e são muito usados em programação.

se toda string é uma sequência de caracteres, você pode acessá-los através de sua posição **sempre contando a partir do zero**, usando o nome da variável junto de colchetes e o index (posição) da letra

```
terceira_letra = nome[1] # "pedro"
print(f"\nsegunda letra de {nome} é '{terceira_letra}'")

terceira_letra = nome[-1] # "pedro"
print(f"\última letra de {nome} é '{terceira_letra}'")
```

```
segunda letra de pedro é 'e'

última letra de pedro é 'o'
```

se usar -1 ele pega o último item automaticamente.

strings e suas operações básicas

tipos possuem **métodos** específicos, eles são funções internas que realizam uma ação com o valor daquela variável:

.strip(): remove os espaços em branco e quebras de linha das extremidades

.lower(): transforma todos os caracteres de uma string em minúsculos. (nice)

.upper(): transforma todos os caracteres de uma string em maiúsculos.

.capitalize(): torna a primeira letra maiúscula.

.title(): torna todas as primeiras letras maiúsculas.

.replace(str, str): recebe duas strings e substitui uma pela outra.

e também a função:

len(str): retorna a quantidade de caracteres da string, isso inclui espaços em branco.

strings também podem ser concatenadas usando o operador aritmético de soma **+**, resultando na adição da nova string ao final da original.

exemplos de operaçoes com string

```python
# métodos de string
frase = "because ignorance is a bliss.\n"

print(f'\nfrase original: {frase}')

print(f'frase maiúscula: {frase.upper()}')

print(f'frase minúscula: {frase.lower()}')

print(f'frase capitalizada: {frase.capitalize()}')

print(f'frase com título: {frase.title()}')

print(f'frase sem quebra de linha: {frase.strip()}')

print(f'frase trocada: {frase.replace("python", "java")}')

# funçoes uteis com string
tamanho_da_frase = len(frase)
print(f'\ntamanho da frase: {tamanho_da_frase} caracteres')

# concatenando string
nova_frase = frase.strip() + " - Gray, Thomas."
print(f'\nfrase concatenada: {nova_frase}')
```

```
frase original: because ignorance is a bliss.

frase maiúscula: BECAUSE IGNORANCE IS A BLISS.

frase minúscula: because ignorance is a bliss.

frase capitalizada: Because ignorance is a bliss.

frase com título: Because Ignorance Is A Bliss.

frase sem quebra de linha: because ignorance is a bliss.
frase trocada: because ignorance is a bliss.


tamanho da frase: 30 caracteres

frase concatenada: because ignorance is a bliss. - Gray, Thomas.
```

números e suas operações básicas

```python
# int e float
produto_preco = 49.99
quantidade = 3
desconto = 5.0

# cálculos básicos
total_bruto = produto_preco * quantidade
total_com_desconto = total_bruto - desconto

print(f"Total Bruto: R$ {total_bruto:.2f}")
print(f"Desconto aplicado: R$ {desconto}")

# aumentando o desconto em 2.0
desconto += 2.0
print(f"\nDesconto atualizado: R$ {desconto}")

#diminuindo a quantidade em 1
quantidade -= 1
print(f"Quantidade atualizada: {quantidade}")

# truques de divisão
print(f"\nDivisão Comum (10 / 3): {10 / 3}")       # 3.333...
print(f"Divisão Inteira (10 // 3): {10 // 3}")     # 3
print(f"Resto da Divisão (10 % 3): {10 % 3}")      # 1

# funções úteis
nota = 8.7654
print(f"\nNota arredondada: {round(nota, 2)}")
```

```
Total Bruto: R$ 149.97
Desconto aplicado: R$ 5.0

Desconto atualizado: R$ 7.0
Quantidade atualizada: 2

Divisão Comum (10 / 3): 3.3333333333333335
Divisão Inteira (10 // 3): 3
Resto da Divisão (10 % 3): 1

Nota arredondada: 8.77
```

interagindo com o usuário

hora de obter o valor das variáveis diretamente com o usuário do programa. pra isso existe a função **input()**, semelhante ao print ela é nativa do python e permite o uso do terminal para interagir com o humano.

input() solicita ao usuário uma entrada no terminal, a aplicação será pausada até que um valor seja inserido.

opcionalmente você pode passar uma **string** como parâmetro, ela será exibida no terminal quando a solicitação de entrada for efetuada.

input() recebe o valor e o **print()** exibe.

crie duas variáveis, numA e numB e receba os valores delas através do **input()**, exiba os valores com **print()**.

```python
# input()
nome = input("qual é o seu nome? ")
print(f"\nolá, {nome}!")

print("qual é sua idade?")
idade = input()
print(f"\nvocê tem {idade} anos.")
```

```
qual é o seu nome? jc

olá, jc!
qual é sua idade?
33

você tem 33 anos.
```

usando input()

resultado de dois valores recebidos através do input()

interagindo_com_usuário.py > ...

```python
# utilizando o input
numA = input("insira o valor do número 1: ")
numB = input("insira o valor do número 2: ")
print(f"\nnumA: {numA}")
print(f"numB: {numB}")
```

```
insira o valor do número 1: 5
insira o valor do número 2: 5

numA: 5
numB: 5
```

usando os dados de input()

ainda com o código anterior vamos fazer uma operação aritmética usando nossas variáveis:

tente realizar **duas** operações, a primeira delas sendo uma **soma** e exiba o resultado no terminal usando **print().**

falhando miseravelmente

ambos os resultados não são conforme esperado.

```python
# utilizando o input
numA = input("insira o valor do número 1: ")
numB = input("insira o valor do número 2: ")
print(f"\nnumA: {numA}")
print(f"numB: {numB}")

# somando os valores
soma = numA + numB
print(f"\nsoma (concatenação): {soma}")

# multiplicando os valores
multiplicacao = numA * numB
print(f"\nmultiplicação: {multiplicacao}")
```

```
insira o valor do número 1: 5
insira o valor do número 2: 5

numA: 5
numB: 5

soma (concatenação): 55
Traceback (most recent call last):
  File "c:\Users\pedro\aulas\tempCodeRunnerFile.py", line 12, in <module>
    multiplicacao = numA * numB
                    ~~~~~~^~~~~~
TypeError: can't multiply sequence by non-int of type 'str'
```

é culpa do input()

o motivo é simples: tudo que o input recebe é tratado como **string**.

com isso nossa soma acabou se tornando uma concatenação e a multiplicação se torna impossível entre letras.

```python
# utilizando o input
numA = input("insira o valor do número 1: ")
numB = input("insira o valor do número 2: ")
print(f"\nnumA: {numA}")
print(f"numB: {numB}")

#verificando os tipos
print("numA é do tipo:", type(numA))
print("numB é do tipo:", type(numB))
```

```
insira o valor do número 1: 5
insira o valor do número 2: 5

numA: 5
numB: 5
numA é do tipo: <class 'str'>
numB é do tipo: <class 'str'>
```

a solução: type casting

pra resolver esse problema existe o **type casting**, é o processo de transformação de textos em valores numéricos e vice-versa. ele acontece usando funções específicas para cada tipo:

- ➔ **int():** Tenta transformar o valor em um número inteiro.
- ➔ **float():** Tenta transformar o valor em um número decimal.
- ➔ **str():** Transforma qualquer valor em texto.

você pode fazer o **type casting** de forma combinada com a função **input()** ou em conversão posterior.

type castings

type cast combinado, o valor é automaticamente convertido ao ser lido do terminal.

```python
# type casting combinado com input()

numA = int(input("insira o valor de numA? "))
print(f'\nnumA: {numA}, tipo: {type(numA)}')
```

```
insira o valor de numA? 5

numA: 5, tipo: <class 'int'>
```

type cast posterior, a leitura é feita, o valor armazenado e a conversão ocorre depois. pode ser útil quando o valor original deve ser mantido.

```python
# type casting posterior
numB = input("insira o valor de numB? ")
numB_float = float(numB)
print(f'\nnumB: {numB_float}, tipo: {type(numB_float)}')
```

```
insira o valor de numB? 5

numB: 5.0, tipo: <class 'float'>
```

```python
# somando os valores
soma = numA + numB_float
print(f'\nsoma: {soma}, tipo: {type(soma)}')
```

```
soma: 10.0, tipo: <class 'float'>
```

troca equivalente

a conversão com type casting só pode acontecer entre valores **válidos.**

uma conversão de um valor só acontece se o conteúdo original é equivalente ao da conversão. as conversões abaixo por exemplo, não podem ocorrer:

```
texto = "vinte"
int(texto)  # gera erro ValueError
float(texto)  # gera erro ValueError
```

```
string = "False"
bool(string)  # sempre retorna True
```

```
texto2 = "1.8"
int(texto2)  # gera erro ValueError
```

javascript é louco

em javascript string tem precedencia na soma e numero tem precedencia na subtracao, e booleanos na verdade são números 😮

```javascript
//string ganha
let a = 1;
let b = "1";
console.log(`\na soma ${a} (${typeof a}) + ${b} (${typeof b}) = ${a + b} (${typeof (a + b)})`);

//number ganha
let c = "5"
let d = 2
console.log(`\na subtração ${c} (${typeof c}) - ${d} (${typeof d}) = ${c - d} (${typeof (c - d)})`);

//string ganha
let e = "10"
let f =  "10"
console.log(`\nna soma ${e} (${typeof e}) + ${f} (${typeof f}) = ${e + f} (${typeof (e + f)})`);

//number ganha
let g = "10"
let h =  "10"
console.log(`\nna subtração ${g} (${typeof g}) - ${h} (${typeof h}) = ${g - h} (${typeof (g - h)})`);

//boolean sao numeros
console.log(true + true);   // 2
console.log(true + false);  // 1
console.log(false + false); // 0
```

```
a soma 1 (number) + 1 (string) = 11 (string)

a subtração 5 (string) - 2 (number) = 3 (number)

na soma 10 (string) + 10 (string) = 1010 (string)

na subtração 10 (string) - 10 (string) = 0 (number)
2
1
0
```

exercicios

tenta fazer ai po

padronizado nomes

crie um programa que recebe um nome como entrada do usuário. seu código deve exibir o nome corretamente formatado no terminal, removendo espaços em branco, letras incorretamente maiúsculas ou minúsculas, etc.

exemplos de entrada:

- joÃO SILVA!
- pEDRO paiva

```
insira o nome: joÃO  SILVA
seu nome corrigido: João  Silva
```

```
insira o nome:    pEDRO paiva
seu nome corrigido: Pedro Paiva
```

idade em dias

solicite ao usuário sua idade e nome.

exiba no terminal uma mensagem formatada contendo uma mensagem de boas vindas ao usuário e sua idade convertida em dias.

conversor de celsius para fahrenheit

receba uma temperatura em graus celsius e faça a conversão dela para graus fahrenheits e exiba uma mensagem formatada com o resultado no terminal.

calculadora IMC

solicite ao usuário seu **peso** e **altura**. realize o cálculo do imc e exiba o valor encontrado no terminal.

dividindo a conta

um grupo de amigos saíram para jantar na brotheragem, a conta a ser pago deve ser dividida igualmente para o grupo. solicite o valor total a ser pago, a quantidade de pessoas no grupo e a taxa adicional a ser paga ao garçom (roubo).

exiba o resultado formatado com o valor que cada um deve pagar.

o valor a ser pago pelas x pessoas é de r$xx,xx

calculando área de retângulos e quadrados

receba como entrada os valores de largura e comprimento de terrenos **quadrados** ou **retangulares**.

calcule a área total do terreno e exiba no terminal.

solicite o valor por metro quadrado do terreno e use para calcular o valor total do terreno.

ao final exiba o **tamanho do terreno** e **valor total** com 2 casas decimais no terminal.

gerador de emails da firma

te contrataram para automatizar o rh do supermarket. sua primeira tarefa é gerar os e-mails corporativos dos funcionários.

solicite do usuário o nome, sobrenome e setor do funcionário.

gere o e-mail seguindo a regra: *nome.sobrenome_setor@supermarket.com.*

exiba o e-mail criado em uma mensagem no terminal.

dica: use concatenação de strings.

fechamento de caixa

solicite o nome do produto, valor e a quantidade. calcule o subtotal e exiba no terminal, insira em seguida o valor pago pelo cliente.

faça o cálculo do troco e exiba uma mensagem formatada contendo um agradecimento, o nome do produto em letras maiúsculas, quantidade comprada e o valor de troco.

calculando comunicação no espaço 🛰️

o sinal de comunicação entre a terra e uma sonda espacial viaja na velocidade da luz (300.000km/s).

solicite como entrada a **distância** em quilômetros de uma sonda enviada para marte, calcula o tempo em **minutos** e **segundos** que uma mensagem leva para chegar até a sonda.

dica: use // para os minutos e % para segundos.